

WEEK 1 OF 3 – LIVE NOW

GEN AI ARCHITECTS TRAINING

Large Language Models

AUDIENCE

Gen AI Architects

30TS

01 Core Definitions

Foundational vocabulary every Gen AI Architect must command

CORE

01

Large Language Model (LLM)

A neural network trained on massive text corpora to predict the next token in a sequence. LLMs learn statistical patterns across billions of parameters, enabling them to generate, summarize, translate, reason, and converse in natural language. Scale—in parameters, data, and compute—is the key driver of emergent capabilities.

CORE

02

Token

The atomic unit of LLM input/output. A token is typically ~ 0.75 words (e.g., "unbelievable" $\rightarrow$ ["un", "believ", "able"]). LLMs operate entirely in token space: they receive a sequence of token IDs, and output probability distributions over the vocabulary at each step. Context windows are measured in tokens.

ARCHITECTURE

03

Transformer

The dominant neural architecture for LLMs (Vaswani et al., 2017). A transformer uses stacked self-attention + feedforward layers. The attention mechanism lets every token attend to every other token in the context window, capturing long-range dependencies

that RNNs struggled with. All modern LLMs (GPT, Claude, Gemini, Llama) are Transformer-based.

ARCHITECTURE

04

Self-Attention

The core mechanism of a Transformer. For each token, Query (Q), Key (K), and Value (V) vectors are computed. Attention scores = $\text{softmax}(QK^T/\sqrt{d_k}) \times V$. This allows each position to dynamically "attend" to the most relevant positions in the input, enabling parallel computation and context-sensitivity simultaneously.

ARCHITECTURE

05

Context Window

The maximum number of tokens an LLM can process in a single forward pass—both input and output combined. A 128K context window means the model can "see" ~96,000 words at once. Longer contexts enable document analysis, long conversations, and multi-step reasoning but increase compute quadratically in naive attention.

ARCHITECTURE

06

Parameters

The learnable weights of a neural network, shaped by training. A 70B model has 70 billion floating-point numbers storing its "knowledge." Parameters live in embedding matrices, attention weight matrices (W_Q, W_K, W_V, W_O), and feedforward layers. More parameters generally mean more capacity, but inference cost scales accordingly.

TRAINING

07

Pre-training

The initial, large-scale training phase where an LLM learns from a massive corpus (Common Crawl, books, code, Wikipedia) via **next-token prediction**. No labels needed—the text itself provides the training signal. Pre-training is compute-intensive (thousands of GPUs, weeks to months) and produces the base model.

TRAINING

08

Fine-tuning

Continued training of a pre-trained model on a smaller, task-specific dataset. **Supervised Fine-Tuning (SFT)** adapts the base model to follow instructions. **Parameter-Efficient Fine-Tuning (PEFT)** methods like LoRA update only a fraction of weights, reducing GPU cost dramatically while preserving most performance gains.

TRAINING

09

RLHF

Reinforcement Learning from Human Feedback. After SFT, human raters rank model outputs. A separate **reward model** is trained on these preferences. The LLM is then optimized with PPO (Proximal Policy Optimization) to maximize reward. RLHF is the key technique that turns raw LLMs into helpful, harmless assistants (used in ChatGPT, Claude).

CONCEPT

10

Embeddings

Dense vector representations of tokens in high-dimensional space (~768–4096 dims). Semantically similar words map to nearby vectors. LLMs use **learned embeddings** as input; these are continuously transformed by attention layers. Embeddings are also extracted as outputs to power semantic search, clustering, and RAG (next week's topic).

CONCEPT

11

Temperature & Sampling

Inference controls for randomness. `Temperature` scales the logits before softmax: $T \rightarrow 0$ is deterministic (argmax), $T \rightarrow 2$ is chaotic. `Top-p (nucleus) sampling` samples from the smallest token set whose cumulative probability exceeds p . `Top-k` restricts sampling to k most likely tokens. These parameters tune creativity vs. consistency.

EVALUATION

12

Hallucination

When an LLM generates `confident but factually incorrect` output. Caused by gaps between training data and reality, the model optimizing for fluency over accuracy, and the probability-based generation process. Mitigation strategies include RAG (grounding), prompt engineering, tool use, and calibration techniques. A critical risk for production architectures.

CONCEPT

13

Prompt Engineering

The practice of crafting inputs (prompts) to elicit desired outputs from LLMs. Key techniques: `few-shot prompting` (examples in context), `chain-of-thought` (asking the model to reason step-by-step), `system prompts` (setting behavior and persona), and `structured output prompting` (JSON, XML). Architects must master this before building on LLMs.

ARCHITECTURE

14

Positional Encoding

Since Transformers process all tokens in parallel (unlike RNNs), they need an explicit signal for token order. `Sinusoidal encodings` (original Transformer) or `RoPE / ALiBi` (modern LLMs) add position information to embeddings. `RoPE` (Rotary Position Embedding) is used by Llama, Mistral, and most open-source models for good length generalization.

15

TRAINING

Scaling Laws

Empirical relationships (Kaplan et al., Hoffmann et al.) showing that LLM performance improves predictably with model size (N), dataset size (D), and compute (C). The Chinchilla scaling law (2022) found optimal training requires roughly 20 tokens per parameter—reshaping how all modern LLMs are trained.

16

EVALUATION

Benchmarks

Standardized tests for measuring LLM capability: MMLU (57-subject knowledge), HumanEval (code generation), HellaSwag (commonsense reasoning), GSM8K (grade-school math), HELM (holistic evaluation). Architects use benchmarks to compare models and select the right one for a use case.

02 Key Algorithms

The mathematical engines that power modern LLMs

Scaled Dot-Product Attention

CORE TRANSFORMER OPERATION

The foundational algorithm. Three projection matrices transform input tokens into Query, Key, and Value vectors. The similarity between Q and K determines how much each V contributes to the output—scaled by vd_k to prevent gradient vanishing in high dimensions.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Multi-Head Attention

MHA

Runs h attention functions in parallel, each with its own learned $Q/K/V$ projections. Each "head" captures different relationship types (syntactic, semantic, positional). Outputs are concatenated and projected. Most LLMs use 32-128 heads. Grouped-Query Attention (GQA) is a more efficient modern variant.

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Byte-Pair Encoding

BPE TOKENIZATION

The dominant tokenization algorithm. Starts with character-level vocabulary and iteratively merges the most frequent adjacent byte pairs. Produces a ~50K token vocabulary balancing rare-word handling (character fallback) with efficiency (common words = 1 token). Used by GPT-4, Claude, Llama, and most modern LLMs.

$$\text{Merge}(A,B) \rightarrow AB \text{ if } \text{freq}(AB) = \max(\text{all pairs})$$

Proximal Policy Optimization

PPO (RLHF PHASE)

The RL algorithm used to align LLMs with human preferences. Clips the policy update ratio to prevent large destabilizing steps. The LLM is the "policy"; human preferences are the "reward signal." A KL-divergence penalty against the reference model prevents reward hacking and catastrophic forgetting.

$$L(\theta) = E[\min(r(\theta)\hat{A}, \text{clip}(r(\theta), 1\pm\epsilon)\hat{A})]$$

Low-Rank Adaptation

LORA

PEFT technique that freezes original weights and injects trainable low-rank matrices ($A \in \mathbb{R}^{(d \times r)}$, $B \in \mathbb{R}^{(r \times k)}$) into attention layers. With rank $r \ll d$, only 0.1-1% of parameters are trained, reducing GPU memory by 3-10x. QLoRA further quantizes the frozen weights to 4-bit for consumer GPU fine-tuning.

$$W' = W_0 + \Delta W = W_0 + BA, \text{ rank}(r) \ll d$$

Flash Attention

IO-AWARE ATTENTION

Algorithmic rewrite of attention that avoids materializing the full $N \times N$ attention matrix in HBM (slow GPU memory). Tiles Q/K/V into blocks processed in fast SRAM, computing attention incrementally. Achieves identical output in $O(N)$ memory vs $O(N^2)$, enabling practical long-context inference. Used by virtually all modern serving stacks.

Tiling: process blocks in SRAM, $O(N^2/M)$ HBM reads

Beam Search

DECODING STRATEGY

A structured decoding algorithm that maintains k "beams" (candidate sequences) at each step, expanding each with the top- k next tokens, then pruning back to k sequences by cumulative log-probability. More coherent than greedy but less diverse than sampling. Commonly used in translation and summarization tasks.

keep top- k : $\text{argmax}_k \sum \log P(t_i | t_1 \dots t_{i-1})$

Rotary Position Embedding

ROPE

Encodes position by rotating Q and K vectors by an angle proportional to their position. The dot product Q^TK then depends only on relative position—not absolute—enabling strong length generalization. Adopted by Llama, Mistral, Falcon, Phi, and most open-source models since 2023. YaRN and LongRoPE extend it to longer contexts.

$$f(x, m) = x \cdot e^{(im\theta)}, \theta_k = 1/100000^{(2k/d)}$$

Speculative Decoding

INFERENCE ACCELERATION

A small "draft" model generates k tokens speculatively. The large "verifier" model evaluates all $k+1$ tokens in a single forward pass and accepts/rejects each. Accepted tokens require no extra cost; rejected tokens fall back to the verifier's sample. Achieves 2-4x speedup with identical output distribution to the target model.

$$\text{Accept token } t_i \text{ if } U \leq P_{\text{target}}(t_i) / P_{\text{draft}}(t_i)$$

03 Weekly Session Plan

Monday-Friday full-day schedule for Gen AI Architects

09:00 - 09:30

LECTURE

Course Overview & AI Architecture Landscape

Series roadmap, learning objectives, tool setup. Where LLMs fit in the modern AI stack.

09:30 - 11:00

LECTURE

What is an LLM? History & Intuition

- N-gram → RNN → Transformer evolution
- Pretraining objective: next-token prediction
- Scale and emergent abilities
- Model families: GPT, Claude, Gemini, Llama

11:00 - 11:15

BREAK

Break

11:15 - 12:30

LECTURE

Tokenization Deep Dive

- BPE, WordPiece, SentencePiece
- Vocabulary size trade-offs
- Counting tokens for cost estimation
- Tokenization edge cases (code, multilingual)

13:30 - 17:00

LAB

Lab 1: LLM API Explorer

First contact with LLM APIs. Tokenize text, inspect probabilities, compare model families.

09:00 - 10:30

LECTURE

Transformer Architecture In Depth

- Encoder vs Decoder vs Encoder-Decoder
- Multi-head attention mechanics
- Layer normalization, residual connections
- FFN, MoE (Mixture of Experts)

10:30 - 11:00

LECTURE

Positional Encoding & RoPE

Sinusoidal, learned, RoPE, ALiBi. Why position matters and how modern LLMs handle long contexts.

FOUNDATIONS

11:15 - 12:30

LECTURE

Context Windows & KV-Cache

How context is managed at inference time. KV-caching mechanics. Sliding window attention. Grouped Query Attention (GQA).

13:30 - 17:00

LAB

Lab 2: Attention Visualization

Visualize attention heads. Compare behavior across layers. Identify what different heads attend to.

09:00 - 10:30

LECTURE

Pre-training: Data, Objectives & Scale

- Training data: quality, deduplication, filtering
- Cross-entropy loss, perplexity
- Scaling laws (Chinchilla)
- Training infrastructure: FSDP, tensor parallelism

10:45 - 12:30

LECTURE

Alignment: SFT, RLHF & DPO

- Why alignment matters for architects
- Supervised Fine-Tuning (SFT)
- RLHF pipeline: reward model + PPO
- DPO: simpler alternative to RLHF
- Constitutional AI (Anthropic)

Tuesday

13:30 - 17:00

LAB

Lab 3: Fine-tune with LoRA

Fine-tune a 7B model using QLoRA on a custom dataset. Compare base vs instruction-tuned behavior. Measure perplexity changes.

09:00 - 10:00

LECTURE

decoding for structured output.

10:00 - 11:15

LECTURE

Prompt Engineering for Architects

- Zero-shot, few-shot, chain-of-thought
- System prompts & persona design
- Structured outputs (JSON mode)
- Prompt injection & jailbreak awareness

11:30 - 12:30

LECTURE

LLM Evaluation & Benchmarks

MMLU, HumanEval, MT-Bench, LLM-as-Judge. Hallucination detection. Building evals for your use case.

13:30 - 17:00

LAB

Lab 4: Prompt Engineering Battle

Teams compete to achieve highest accuracy on a benchmark task using only prompt engineering. Techniques judged on zero-shot and few-shot performance.

09:00 - 10:30

LECTURE

Production LLM Architecture

- Model serving: vLLM, TGI, Triton
- Quantization: INT8, GPTQ, AWQ
- Speculative decoding for throughput
- Cost modeling: tokens × \$/1M

10:45 - 12:00

LECTURE

Model Selection & API Patterns

Choosing between GPT-4o, Claude, Gemini, Llama. API patterns: streaming, batching, tool/function calling. Cost vs quality trade-offs.

13:00 - 15:30

LAB

Lab 5: Build a Production LLM Pipeline

End-to-end: ingest text → tokenize → call LLM API with streaming → structured JSON output → evaluation harness. Deploy to a serverless endpoint.

INFERENCE

15:30 - 17:00

ASSESSMENT

Week 1 Review & Architecture Showcase

Teams present their pipelines. Q&A on concepts. Preview of Week 2: Retrieval-Augmented Generation.

04

Lab Guides

Hands-on exercises – one per day, progressively building toward a production pipeline

MON



Lab 1: LLM API Explorer

→ Get hands-on with LLM APIs; understand tokenization, probabilities, and model differences

1	Environment Setup Install Python env, set API keys for OpenAI + Anthropic. Verify connectivity with a "Hello, world" prompt to both.
---	--

Friday

DAY 5 OF 5

ARCHITECTURE DAY

2	Tokenize & Count Use tiktoken (OpenAI) and claude tokenizer to tokenize the same 5 texts. Compare token counts. Estimate costs for 1M similar documents.
3	Logprobs Exploration Enable logprobs on GPT-4o. Ask ambiguous questions. Visualize top-5 token probabilities at each step. Identify where the model is uncertain.
4	Model Comparison Run identical prompts through GPT-4o-mini, Claude Haiku, and Llama 3.1 8B. Compare latency, cost, and output quality on 3 tasks: summarization, code, reasoning.
5	Deliverable Markdown report: token count table, cost projection, model comparison scorecard. Share with group Slack channel.

PYTHON 3.11

OPENAI SDK

ANTHROPIC SDK

TIKTOKEN

JUPYTER

TUE

Lab 2: Attention Visualization

→ Develop intuition for how attention mechanisms work inside a Transformer

1	Load a Small Model Load GPT-2 (124M) locally via HuggingFace Transformers. Enable output_attentions=True in a forward pass.
2	Extract Attention Weights Extract all 12×12 attention matrices (12 layers × 12 heads). Shape: [layers, heads, seq_len, seq_len].
3	Heatmap Visualization Plot attention heatmaps using BertViz or seaborn. Identify heads that attend to: pronouns, next-word, syntactic heads, positional patterns.
4	Head Ablation Zero out individual heads and measure perplexity change. Which heads are most important? Compare across different sentence types.
	Deliverable

5	Annotated notebook with 6 attention heatmaps and written interpretations of what each head pattern represents.
---	--

- HUGGINGFACE TRANSFORMERS
- BERTVIZ
- PYTORCH
- SEABORN
- GPT-2

WED

Lab 3: Fine-tune with QLoRA

→ Adapt a 7B open-source LLM to a custom task using parameter-efficient fine-tuning

1	Dataset Preparation Format 500 instruction-response pairs in Alpaca format. Split 80/10/10 train/val/test. Compute baseline perplexity of Mistral-7B on test set.
2	Configure QLoRA Load Mistral-7B in 4-bit NF4 with bitsandbytes. Configure LoRA: r=16, alpha=32, target modules=[q_proj, v_proj]. Count trainable parameters.
3	Fine-tune with TRL Use HuggingFace TRL SFTTrainer. Train for 3 epochs on single A100. Monitor train/val loss curves. Save adapter weights (~100MB vs ~14GB full model).
4	Evaluate Load merged model. Run test set. Compare: base perplexity vs fine-tuned perplexity. Run 10 qualitative comparisons and score 1-5 per output.
5	Deliverable Training run dashboard (W&B), loss curves, perplexity comparison table, 5 before/after response examples.

- MISTRAL-7B
- PEFT / LORA
- BITSANDBYTES
- TRL
- W&B
- A100 GPU

THU

Lab 4: Prompt Engineering Battle

→ Master advanced prompting techniques through competitive benchmarking

1	Establish Baseline
---	--------------------

	Run the 50-question test set (mix of reasoning, code, knowledge) with a naive prompt. Record zero-shot accuracy as baseline.
2	Apply Techniques Each team applies: few-shot prompting, chain-of-thought, self-consistency voting, system prompt tuning, and structured output formats. One technique per round.
3	Structured Output Challenge Force the model to output JSON with schema: {reasoning, answer, confidence}. Parse and evaluate programmatically.
4	Adversarial Round Teams try to break each other's system prompts with adversarial inputs. Points for successful jailbreaks AND successful defenses.
5	Deliverable Prompt engineering playbook: each technique tested, accuracy delta, and recommendation for when to use it in production.

GPT-40

CLAUDE 3.5 SONNET

LANGCHAIN

PROMPTFOO

PYTHON



Lab 5: Production LLM Pipeline

→ Build a complete, evaluated, deployable LLM pipeline from scratch

1	Ingestion & Preprocessing Accept PDF/text input. Chunk intelligently (sentence-aware, 512 tokens, 10% overlap). Tokenize and validate within context limits.
2	Streaming API Integration Call Claude or GPT-4o with streaming. Implement SSE (Server-Sent Events) to forward tokens to a React frontend in real time.
3	Structured Output Layer Use JSON mode + Pydantic schema validation. Route outputs to different handlers based on response type (summary, Q&A, code, error).
4	Evaluation Harness Auto-evaluate 20 outputs using LLM-as-Judge (GPT-4o scoring 1-5 on helpfulness, accuracy, safety). Log to a dashboard.
	Deploy & Present

5

Package as FastAPI + Docker. Deploy to a cloud endpoint. Demo live to the group. Share GitHub repo link in Slack.

FASTAPI

DOCKER

PYDANTIC

SSE / STREAMING

LLM-AS-JUDGE

GITHUB ACTIONS

05 Week 1 Outcomes

What Gen AI Architects will be able to do by Friday 5PM



Explain LLMs at Every Level

From C-suite analogy to engineering deep dive. Describe the full pipeline: pre-training → alignment → inference → evaluation. Articulate trade-offs between model families confidently.



Design LLM-Powered Systems

Select the right model for a given cost/quality/latency constraint. Architect streaming pipelines with structured outputs. Know when to fine-tune vs prompt vs switch models.



Run LLM Experiments

Design and execute prompt engineering experiments with proper controls. Fine-tune a 7B model with QLoRA. Build and run automated evaluation harnesses using LLM-as-Judge.



Apply Core Algorithms

Understand and apply: attention mechanics, BPE tokenization, LoRA fine-tuning, RLHF alignment, speculative decoding, and Flash Attention in real system design decisions.



Deploy a Production Pipeline

Ship a Dockerized FastAPI LLM service with streaming, Pydantic output validation, cost tracking, and automated evaluation. Ready for the RAG extension next week.



Manage LLM Risks

Identify and mitigate hallucination, prompt injection, and jailbreak risks. Apply evaluation-driven development. Know when an LLM is the wrong tool entirely.

Gen AI Architects Training Series

Next week: **Retrieval-Augmented Generation (RAG)** – vector DBs, embeddings, chunking strategies, re-ranking, and production RAG pipelines.

CURRENT

Week 1: LLMs

NEXT WEEK

Week 2: RAG

WEEK AFTER

Week 3: Chatbots